

PROJECT

SISTEM OPERASI

ANGGOTA KELOMPOK:

- 
1. **ALFANDI WIJAYA** (232200156)
 2. **ALVERN BRAINARD** (232200346)
 3. **DEVLIN MANUEL** (232202935)
 4. **PETRA GAMMA SETYA AGATHA** (232300115)

IT & BIG DATA ANALYTICS

CALVIN INSTITUTE OF TECHNOLOGY

Assignment 1: LRU Page Fault Simulator

Penjelasan Proses Kode:

```
class LRUPageFaultSimulator:
    def __init__(self, capacity):
        self.capacity = capacity
        self.page_frames = []
        self.page_order = []
        self.page_faults = 0
        self.page_hits = 0
```

Dalam class *LRUPageFaultSimulator* fungsi *__init__(self, capacity)* menginisiasi kapasitas frame yang tersedia, *page_frame* yang berisi page, *page_order* untuk mengetahui page yang sering digunakan dimana page yang paling sering digunakan akan ditempatkan di paling belakang, *page_faults* yang merupakan jumlah akses page, dan *page_hits* yang merupakan jumlah akses page yang sama.

```
def apply_lru_algorithm(self, page):
    lru_page = self.page_order.pop(0)
    self.page_frames.remove(lru_page)
    self.page_frames.append(page)
```

Pada fungsi *apply_lru_algorithm* fungsi akan mengambil page yang paling jarang digunakan lalu menggantinya dengan page baru.

```
def simulate_page_fault(self, page):
    if page in self.page_frames:
        self.page_hits += 1
        self.page_order.remove(page)
        self.page_order.append(page)
        print(f"Page hit for page {page}. Page frames unchanged.")
    else:
        self.page_faults += 1
        if len(self.page_frames) < self.capacity:
            self.page_frames.append(page)
        else:
            self.apply_lru_algorithm(page)
        self.page_order.append(page)
        print(f"Page fault for page {page}. Page frames after fault: {self.page_frames}")
```

Pada method *simulate_page_faults(self, page)* fungsi akan melihat apakah page ada dalam frame, jika page ada dalam frame maka variabel *page_hits* akan ditambah 1 dan memindahkan urutan page dalam *page_order* ke posisi paling belakang. Namun, jika page

tidak ada dalam `page_frame` maka variabel `page_faults` akan bertambah 1 dan memeriksa apakah frame masih tersedia. Jika frame masih tersedia maka page akan langsung dimasukkan, namun jika tidak program akan memanggil fungsi `apply_lru_algorithm` untuk menukar page dalam frame.

```
def run_simulation(self, pages):
    for page in pages:
        self.simulate_page_fault(page)
    print(f"\nTotal page faults: {self.page_faults}")
    print(f"Total page hits: {self.page_hits}")
```

Pada fungsi `run_simulation(self, pages)` setiap page akan disimulasikan dan pada akhirnya akan mengembalikan total page faults dan total page hits.

```
# Contoh penggunaan
if __name__ == "__main__":
    capacity = int(input("Enter the number of page frames: "))
    page_sequence = input("Enter the page sequence (space-separated): ").split()
    simulator = LRUPageFaultSimulator(capacity)
    simulator.run_simulation(page_sequence)
```

Pada fungsi `main`, user akan diminta untuk memasukkan jumlah frame dan page apa saja. Lalu program akan mensimulasikan LRU page faults.

Output:

```
Enter the number of page frames: 3
Enter the page sequence (space-separated): A B C A B D A B C D
Page fault for page A. Page frames after fault: ['A']
Page fault for page B. Page frames after fault: ['A', 'B']
Page fault for page C. Page frames after fault: ['A', 'B', 'C']
Page hit for page A. Page frames unchanged.
Page hit for page B. Page frames unchanged.
Page fault for page D. Page frames after fault: ['A', 'B', 'D']
Page hit for page A. Page frames unchanged.
Page hit for page B. Page frames unchanged.
Page fault for page C. Page frames after fault: ['A', 'B', 'C']
Page fault for page D. Page frames after fault: ['B', 'C', 'D']

Total page faults: 6
Total page hits: 4
```

Assignment 2: File Integrity Checker

Penjelasan Proses Kode:

```
import os
import hashlib
import json
import argparse
```

Awal dari kode ini adalah mengimport library pada python yaitu **JSON** yang memungkinkan membaca data dalam format JSON, lalu library **HASHLIB** yang digunakan untuk melindungi data penting misalnya seperti *password* maka library ini akan men-*decode* *password* menjadi suatu kode tertentu sehingga tidak diketahui kode lain. Berikutnya ada library **ARGPARSE** yang memungkinkan pengguna untuk memberikan input melalui *command-line*, seperti menambah data baru, melihat data, ataupun menghapus data. Selanjutnya ada library **OS** yang berfungsi untuk memeriksa dan mengelola file, lalu memastikan file untuk dapat diakses dan di update.

```
class FileIntegrityChecker:
    def __init__(self, hash_algo="sha256"):
        self.hash_algo = hash_algo
        self.hash_func = getattr(hashlib, hash_algo)

    def compute_hash(self, file_path):
        try:
            with open(file_path, "rb") as f:
                file_hash = self.hash_func()
                while chunk := f.read(4096):
                    file_hash.update(chunk)
            return file_hash.hexdigest()
        except Exception as e:
            print(f"Error computing hash for {file_path}: {e}")
            return None
```

Pada bagian gambar diatas, kami menginisiasi kelas *FileIntegrityChecker* dengan memilih algoritma hash yang diinginkan. Jika tidak ada, algoritma default yang digunakan adalah SHA-256. Berikutnya ada function *compute_hash* digunakan untuk menghitung nilai hash dari file, dan nilai hash yang dihitung dapat dibandingkan dengan nilai hash referensi untuk memverifikasi apakah file tersebut telah berubah.

```
def scan_directory(self, directory):
    file_hashes = {}
    for root, _, files in os.walk(directory):
        for file in files:
            file_path = os.path.join(root, file)
            file_hash = self.compute_hash(file_path)
            if file_hash:
                file_hashes[file_path] = file_hash
    return file_hashes
```

Method kedua dari kelas *FileIntegrityChecker* adalah `scan_directory`. Method ini berfungsi untuk menghitung hash setiap file di folder/direktori sehingga dapat membantu mengenali file berdasarkan isinya, bukan hanya nama filenya. Method ini juga berguna untuk mendeteksi perubahan pada file atau mengidentifikasi duplikasi file. Selain itu, kita dapat mendeteksi apakah file tertentu telah dimodifikasi. Method ini akan menggabungkan direktori saat ini (`root`) dengan nama file untuk mendapat jalur file. Kemudian `self.compute_hash(file_path)` akan dipanggil untuk menghitung hash file berdasarkan isi file tersebut. Pada akhirnya, method ini akan mengembalikan semua file dan hashnya masing-masing di direktori.

```
def save_hashes(self, file_hashes, output_file):
    with open(output_file, "w") as f:
        json.dump(file_hashes, f, indent=4)
```

Method `save_hashes` dari kelas *FileIntegrityChecker* akan menyimpan hasil output terbaru ke dalam sebuah file. Hasil output tersebut berupa hash file beserta jalur file-nya yang akan disimpan ke dalam sebuah file output dalam format JSON. Dalam hal ini, file baru akan dibuka dan hash file tersebut akan dituliskan ke dalam file yang ditulis.

```
def load_hashes(self, input_file):
    with open(input_file, "r") as f:
        return json.load(f)
```

Method `load_hashes` dari kelas *FileIntegrityChecker* akan berfungsi untuk membuka hash file beserta jalur filenya yang disimpan dari sebuah file (disebut sebagai `input_file`). Secara lebih spesifik, method akan membuka file JSON untuk dibaca dan akan mengembalikan data tersebut dalam program python terkait yang sedang dijalankan.

```
def verify_files(self, directory, hash_file):
    current_hashes = self.scan_directory(directory)
    stored_hashes = self.load_hashes(hash_file)

    added_files = set(current_hashes.keys()) - set(stored_hashes.keys())
    deleted_files = set(stored_hashes.keys()) - set(current_hashes.keys())
    modified_files = {
        file for file in current_hashes if file in stored_hashes and current_hashes[file] != stored_hashes[file]
    }

    # Verbose report
    print("\nFile Integrity Checker - Verification (Verbose Mode)")
    print("-" * 50)
    print(f"Directory to scan: {directory}")
    print(f"Using hash database: {hash_file}")
    print("\nVerifying files...\n")

    for file in current_hashes:
        if file in added_files:
            print(f"Processing: {file} [New File]")
        elif file in modified_files:
            print(f"Processing: {file} [Modified]")
        else:
            print(f"Processing: {file} [OK]")

    for file in deleted_files:
        print(f"Processing: {file} [Deleted]")

    print("\nVerification Results:")
    print(f"- Total files scanned: {len(current_hashes) + len(deleted_files)}")
    print(f"- Unchanged files: {len(current_hashes) - len(added_files) - len(modified_files)}")
    print(f"- Modified files: {len(modified_files)}")
    print(f"- New files: {len(added_files)}")
    print(f"- Deleted files: {len(deleted_files)}\n")
```

Dalam function *verify_files* memanggil function *scan_directory* yang digunakan untuk memindai semua file dalam direktori yang ditentukan dan menghasilkan nilai hash untuk setiap file. Berikutnya nilai hash yang telah tersimpan akan dimuat dari file yang ada (JSON).

```

def main():
    parser = argparse.ArgumentParser(description="File Integrity Checker")
    parser.add_argument("command", choices=["init", "update", "verify"], help="Action to perform")
    parser.add_argument("--dir", required=True, help="Directory to scan")
    parser.add_argument("--hash_file", default="file_hashes.json", help="File to store/load hashes")
    parser.add_argument("--hash_algo", default="sha256", help="Hash algorithm to use (default: sha256)")

    args = parser.parse_args()
    checker = FileIntegrityChecker(hash_algo=args.hash_algo)

    if args.command == "init":
        hashes = checker.scan_directory(args.dir)
        checker.save_hashes(hashes, args.hash_file)
        print(f"Initialized hash database at {args.hash_file}")

    elif args.command == "update":
        hashes = checker.scan_directory(args.dir)
        checker.save_hashes(hashes, args.hash_file)
        print(f"Updated hash database at {args.hash_file}")

    elif args.command == "verify":
        checker.verify_files(args.dir, args.hash_file)

if __name__ == "__main__":
    main()

```

Ketika akan memulai program, fungsi `main()` akan dijalankan untuk melakukan pemeriksaan *File Integrity* dengan menggunakan hashing. Dalam hal ini, digunakan library *argparse* untuk mem-parsing argumen dari command-line, sehingga program dapat dijalankan dengan berbagai opsi dan perintah. Selanjutnya, perintah-perintah yang ingin dijalankan ditambahkan kepada variabel parser melalui *command* bernama `add_argument`. Kemudian, dibuatlah variabel baru bernama `args` dan `checker`. Checker adalah objek dengan algoritma hashing yang dipilih oleh pengguna (`args.hash_algo`). Objek inilah yang akan membaca direktori, menghitung hash dari masing-masing file, menyimpan dan memuat *hash database*, dan memverifikasi hasil pemeriksaan dari *File Integrity*.

Selanjutnya program akan dijalankan berdasarkan opsi/pilihan perintah yang diberikan. Jika *command* (perintah) nya adalah “init”, maka program akan melakukan pemindaian terhadap direktori dengan menjalankan fungsi `scan_directory()` terhadap objek checker dan menghitung hash dari semua file. Kemudian program akan menyimpan hash yang telah dihitung ke file JSON melalui fungsi `save_hashes()`. Output yang akan dihasilkan dari command ini adalah berupa notifikasi bahwa *hash database* telah dibuat. Jika *command* nya adalah “update”, program akan memperbarui *hash database* dengan menjalankan fungsi sama seperti *command* init, hanya saja *command* update ini hanya dapat dilakukan dan berlaku jika sebelumnya telah dilakukan inisialisasi *hash database*. Secara spesifik, program akan menggantikan hash yang sudah ada di file JSON dengan hash yang baru. Selanjutnya, jika

command nya adalah “verify”, maka program akan memeriksa apakah file dalam direktori sesuai dengan hash yang telah disimpan sebelumnya melalui fungsi bernama `verify_files`. Dalam hal ini, program dapat mendeteksi perubahan file yang tidak sah (jika ada file yang rusak, dan lain sebagainya).

Input yang diberikan kepada program:

```
PS C:\Users\Asus\Documents\Pemrograman\configs> python file_integrity_checker.py verify --dir C:\Users\Asus\Documents\Pemrograman\configs --hash_file file_hashes.json
```

Output:



```
File Integrity Checker - Verification (Verbose Mode)
-----
Directory to scan: C:\Users\Asus\Documents\Pemrograman\configs
Using hash database: file_hashes.json

Verifying files...

Processing: C:\Users\Asus\Documents\Pemrograman\configs\devlin.txt [New File]
Processing: C:\Users\Asus\Documents\Pemrograman\configs\fandi.txt [OK]
Processing: C:\Users\Asus\Documents\Pemrograman\configs\file_hashes.json [Modified]
Processing: C:\Users\Asus\Documents\Pemrograman\configs\file_integrity_checker.py [OK]
Processing: C:\Users\Asus\Documents\Pemrograman\configs\petra.txt [New File]
Processing: C:\Users\Asus\Documents\Pemrograman\configs\file3.txt [Deleted]

Verification Results:
- Total files scanned: 6
- Unchanged files: 2
- Modified files: 1
- New files: 2
- Deleted files: 1

Detailed Report:

Modified Files:
1. C:\Users\Asus\Documents\Pemrograman\configs\file_hashes.json

New Files:
1. C:\Users\Asus\Documents\Pemrograman\configs\petra.txt
1. C:\Users\Asus\Documents\Pemrograman\configs\devlin.txt

Deleted Files:
1. C:\Users\Asus\Documents\Pemrograman\configs\file3.txt
PS C:\Users\Asus\Documents\Pemrograman\configs>
```